School of Future Tech

Case Study Report

on

TIC TAC TOE

by

BHUVISHA GOHIL

150096725190

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

In the modern era of digital entertainment, web-based games are increasingly popular due to their accessibility and ability to run directly in a browser without additional installation. Tic Tac Toe is a classic two-player game that is simple in rules yet provides opportunities to demonstrate logic, strategy, and programming skills.

TTT Arena is an advanced web application that takes the traditional Tic Tac Toe game and expands it into a fully interactive platform. The project incorporates modern UI/UX principles, real-time score tracking, a badge-based ranking system, and a community forum. This allows players to compete, track their achievements, and interact with others, all within a single web interface.

The project leverages **HTML, CSS, and JavaScript** to deliver a responsive and engaging game experience. It demonstrates key web development concepts such as dynamic DOM manipulation, event handling, localStorage usage, and interactive design.

The main objective of this project is to combine gameplay mechanics with user engagement features, creating a platform that is educational, fun, and visually appealing.

2. Problem Statement / Case Background (Abstract)

Most online Tic Tac Toe implementations are extremely basic, offering only the game grid and the ability to play a match. They typically lack features such as:

- Persistent score tracking across sessions
- Badge-based player progression
- Community discussion via forums
- Theme customization

Without these features, users have limited incentive to continue playing or engaging with the platform.

Problem: Build a Tic Tac Toe web application that goes beyond simple gameplay by:

- Tracking wins, losses, and draws in real time
- Awarding badges based on performance
- Offering theme options for visual customization
- Providing a forum for strategy sharing and discussion

Abstract:
This case study presents **TTT Arena**, a browser-based Tic Tac Toe game platform that includes modern gameplay features, leaderboard management, badge-based ranking, and community engagement. It uses **JavaScript** to implement game logic and real-time updates, **HTML** for structured interface design, and **CSS** for styling and responsive layout. The system is fully frontend-based, running entirely in the browser without backend integration.

3. Case Study Design

The application is structured into several modules to ensure clarity and maintainability:

3.1 User Interface Module

The UI is designed to be clean, modern, and intuitive. Components include:

- **Game Board:** A 3×3 grid for the Tic Tac Toe gameplay
- **Scoreboard:** Displays player names, wins, losses, draws, and current badge
- **Turn Indicator:** Shows whose turn it is in real time
- **Game Controls:** Buttons to restart the round, reset scores, or change players
- **Forum Section:** Users can post strategies, like/unlike posts, and filter content
- **Theme Toggle:** Users can switch between Dark, Neon, and Light themes

The design emphasizes **usability, accessibility, and responsiveness**, ensuring the game works on both desktop and mobile devices.

3.2 Game Logic Module

Implemented in **JavaScript**, this module controls all gameplay mechanics:

- Alternates turns between Player X and Player O
- Detects winning patterns after every move
- Handles draw situations when all cells are filled
- Updates scores and badges in real time
- Prevents invalid moves or multiple clicks on the same cell

3.3 Badge and Ranking System

To increase engagement and reward progress:

- Badges range from Newcomer to Grand Master, based on the number of wins
- Badges update automatically on the scoreboard
- Leaderboard ranks players by wins and win rate

- Encourages players to compete and improve their gameplay

3.4 Forum Module

The forum is a fully interactive component designed for community interaction:

- Players can create posts with a title, content, and category tag
- Like/unlike functionality to interact with posts
- Delete functionality for moderation
- Real-time character count for posts
- Filter buttons to view posts by category (Strategy, Wins, Questions, General)

3.5 Theme Management

The platform supports multiple themes:

- **Dark:** Default theme with modern dark colors
- **Neon:** Vibrant neon colors with glowing effects
- **Light:** Clean light theme for a softer appearance
- Themes are saved in localStorage and applied automatically when revisiting

4. Methods & Algorithms Technology Applied in the Problem Statement/ Case Study

4.1 Game Logic Algorithm

The core of TTT Arena relies on pattern recognition for win detection. The game maintains a board array of 9 elements representing the 3×3 grid. Each element is either 'X', 'O', or null. After every move, the algorithm checks for a winning pattern using a predefined list:

Winning Patterns:

1. Rows: **[0,1,2], [3,4,5], [6,7,8]**
2. Columns: **[0,3,6], [1,4,7], [2,5,8]**
3. Diagonals: **[0,4,8], [2,4,6]**

The algorithm iterates through these patterns and compares the values of the board cells. If all three positions in a pattern match the current player, the game highlights the winning cells using a CSS animation and triggers the win logic.

In case the board is full but no winning pattern is found, the game declares a draw and triggers a draw animation. This ensures robust handling of all possible game outcomes.

4.2 Badge & Ranking Algorithm

TTT Arena introduces player progression using a badge system:

- **Newcomer:** 0+ wins
- **Rookie:** 1+ wins
- **Challenger:** 5+ wins
- **Champion:** 10+ wins
- **Legend:** 20+ wins
- **Grand Master:** 50+ wins

The system calculates the badge based on the player's total wins stored in localStorage. The leaderboard ranks players first by total wins, then by win rate (wins/games), providing a fair and competitive scoring mechanism.

4.3 Forum Logic

The forum is implemented entirely on the frontend using JavaScript and localStorage.

- Each post has an ID, author, title, content, tag, likes, and liked status.
- Posts can be liked/unliked, and these interactions update the stored post data instantly.
- Posts are filterable by tag (Strategy, Wins, Questions, General) to enhance navigation and usability.
- Real-time character counting ensures user input remains within limits.

This frontend-only approach demonstrates state management, DOM updates, and dynamic UI rendering without backend support.

4.4 UI & UX Techniques

- **Dynamic Turn Indicator:** Highlights whose turn it is and changes color based on the current player (X or O).
- **Animations:** Winning combinations, pop-ins for X/O marks, and pulsating turn indicators enhance engagement.
- **Theme Toggle:** Changes the visual appearance of the game in real time with Dark, Neon, and Light modes. Themes persist via localStorage.
- **Responsive Design:** CSS Grid and Flexbox are used to ensure the platform works seamlessly across desktop, tablet, and mobile devices.

4.5 Technology Stack

- **HTML5:** Structure of the game, forum, and navigation.

- **CSS3:** Visual styling, animations, hover effects, responsive design.
- **JavaScript (ES6+):** Game logic, leaderboard calculation, badge management, DOM manipulation, event handling.
- **LocalStorage API:** Persisting scores, forum posts, and theme selection.

This combination demonstrates a practical frontend-only architecture for building interactive game platforms without server dependencies.

5. Problem Statement / Case Study Implementation Details and Snapshots.

5.1 Game Board Implementation

- **HTML:** Each cell is a with a unique ID (cell-0 to cell-8).
- **CSS:** Styles include hover effects, animations for X and O, and highlighted winning lines.
- **JavaScript:** makeMove(i) handles user clicks, validates the move, updates the board state, and switches turns.

5.2 Win and Draw Detection

- The endGame(pattern) function identifies the winner or draw.
- Winning cells are highlighted using the winning CSS class and a line is drawn across the winning combination with SVG (<line id="win-line">).
- Dynamic banners and modals display the result with celebratory messages, adding to the immersive experience.

5.3 Leaderboard & Badge System

- Player statistics are saved in localStorage.
- The leaderboard table dynamically renders top players with badges and win rates.
- Badges are calculated based on wins using getBadge(wins).
- The leaderboard supports real-time updates after every game.

5.4 Forum Implementation

- Posts are stored in localStorage as JSON objects.
- Rendering logic (renderPosts) dynamically generates HTML elements for each post, including title, content, and buttons for like/unlike and delete.

- Filter buttons allow users to display posts by category.
- Real-time updates are handled entirely on the frontend without refreshing the page.

5.5 Theme and UI Snapshots

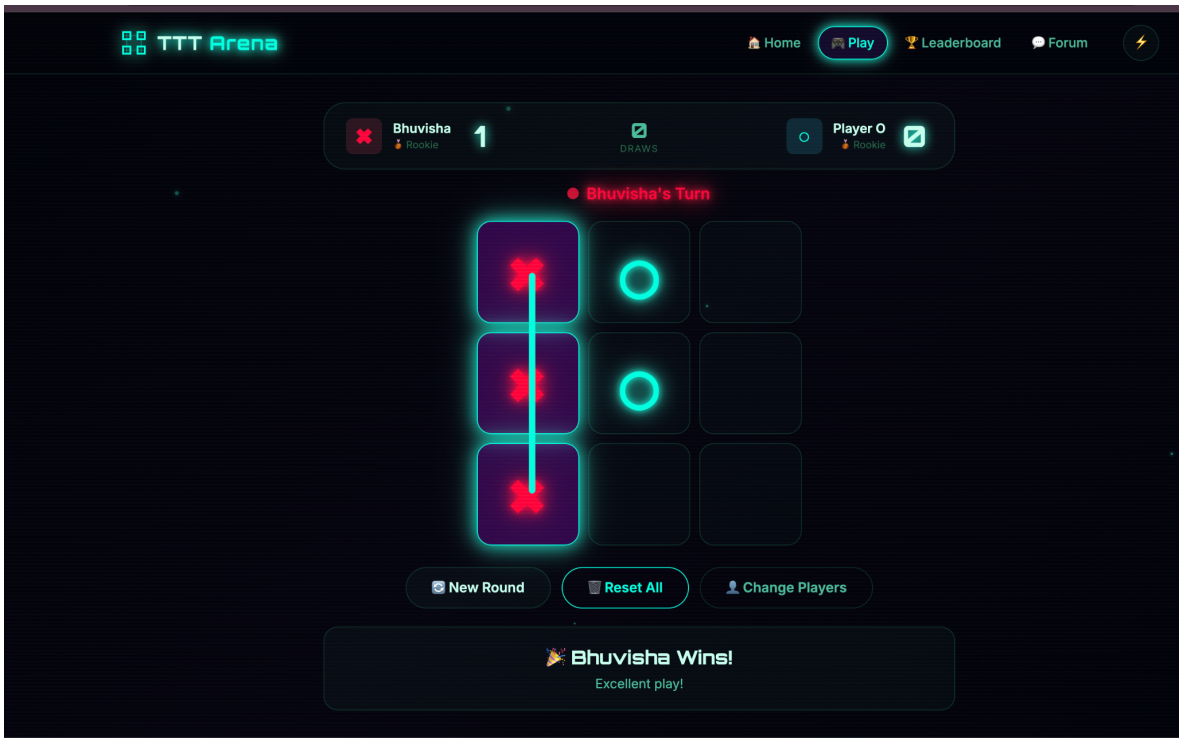
- Themes are switched via a button and saved in localStorage for persistence.
- Neon theme introduces glowing X/O, pulsating turn indicator, and subtle background particles.
- Light theme provides a clean, minimal look with readable typography.

5.6 Game Flow Snapshots

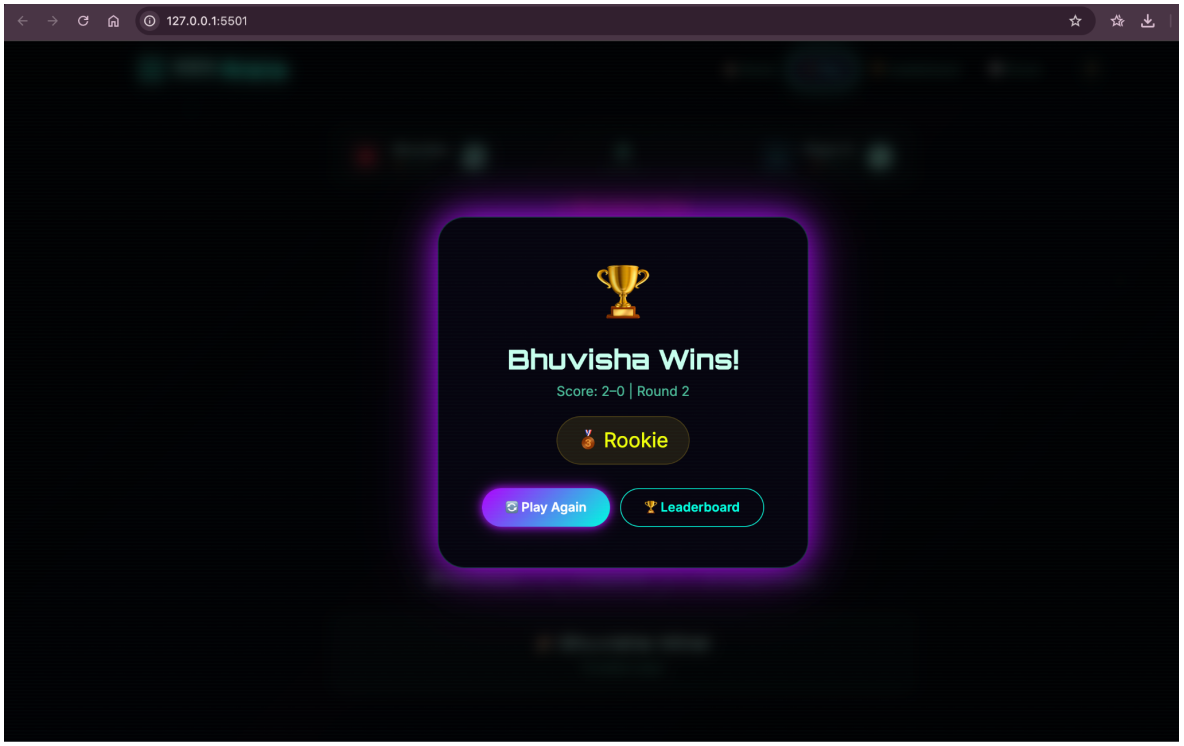
1. **Player Setup:** Users input names and launch the game.
2. **Gameplay:** Real-time turn indicator, click animations, and move validation.
3. **Win/Draw:** Highlighted winning pattern, modal popup with badge, celebratory message.
4. **Leaderboard Update:** Scores updated instantly after every match.
5. **Forum Interaction:** Users post strategies, like posts, and filter content.

Illustrative Snapshots:

1.Game Board: Shows X/O marks and winning line animation

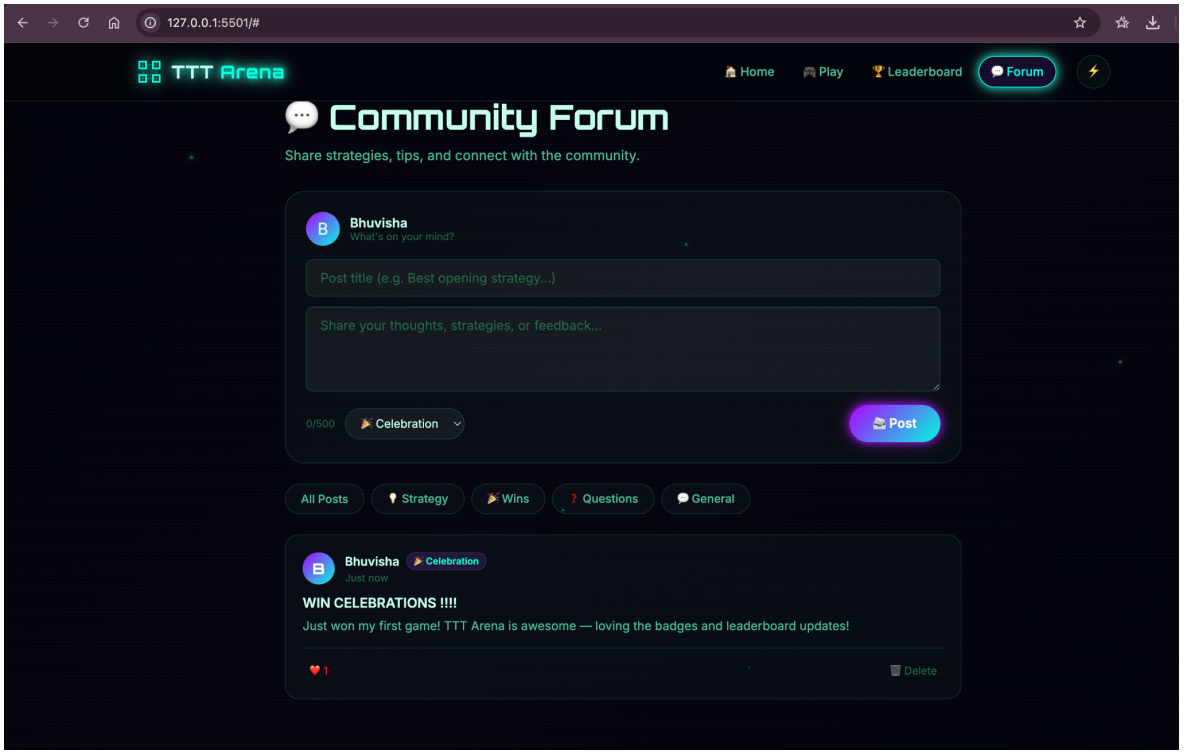


2.Winning Modal: Shows trophy, badge, and dynamic celebratory message



3.Leaderboard Table: Top players with badges and win rates

4.Forum: Example post with likes, tags, and filtering options



6. Problem Statement / Case Study Results and Conclusion.

Results

The **TTT Arena** successfully demonstrates:

- Smooth gameplay with responsive UI
- Real-time winner detection and draw handling
- Persistent leaderboard and badge system
- Interactive forum with filtering and likes
- Theme customization with Dark, Neon, and Light modes

Observations:

- Players can immediately see their progress and achievements
- Forum encourages engagement and knowledge sharing
- Badge system motivates continued play
- The platform is fully frontend, requiring no server-side integration

Conclusion

This project showcases how a simple game can be transformed into a fully interactive, community-driven web platform.
Key takeaways:

- Effective use of **HTML, CSS, and JavaScript** to create an engaging experience
- Implementation of game logic algorithm**s** for real-time win detection
- Use of **l**ocalStorage for persistent user data
- Design of UI/UX elements for usability and engagement

Future Enhancements:

- Adding sound effects for moves, wins, and draws
- Background music toggle
- Option to play against an AI opponent
- User authentication for multi-device progress

7. References

- Mozilla Developer Network (MDN) – JavaScript Documentation
- W3Schools – HTML, CSS, and JavaScript Tutorials
- Stack Overflow – JavaScript DOM Manipulation and Event Handling
- Web Development Best Practices – Responsive Design Guidelines
- Online Tic Tac Toe Logic Examples and Patterns